Análisis de componentes principales simplificado: tutorial paso a paso

Implementar el algoritmo PCA desde cero con Python



Marcus Sena Seguir 9 minutos de lectura · 8 de junio de 2024

207 1



Foto de Volodymyr Hryshchenko en Unsplash

Muchos paquetes de Python consolidados (como scikit-learn) implementan algoritmos de aprendizaje automático, como el algoritmo de Análisis de Componentes Principales (PCA). Entonces, ¿para qué molestarse en aprender cómo funcionan los algoritmos?

Una comprensión profunda de los conceptos matemáticos subyacentes es crucial para tomar mejores decisiones basadas en el resultado del algoritmo y evitar tratarlo como una "caja negra".

En este artículo, muestro la intuición del funcionamiento interno del algoritmo PCA, cubriendo conceptos clave como *reducción de dimensionalidad* , *vectores propios* y *valores propios* , luego implementaremos una clase de Python para encapsular estos conceptos y realizar un análisis PCA en un conjunto de datos.

Ya sea que sea un principiante en aprendizaje automático que intenta desarrollar una comprensión sólida de los conceptos o un profesional interesado en crear aplicaciones de aprendizaje automático personalizadas y necesita comprender cómo funcionan los algoritmos bajo el capó, este artículo es para usted.

. . .

Tabla de contenido

1. Reducción de la dimensionalidad
2. ¿Cómo funciona el análisis de componentes principales?
3. Implementación en Python
4. Evaluación e interpretación
5. Conclusiones y próximos pasos

1. Reducción de la dimensionalidad

Muchos problemas reales de aprendizaje automático involucran conjuntos de datos con miles o incluso millones de características. El entrenamiento de estos conjuntos de datos puede ser computacionalmente exigente, y la interpretación de las soluciones resultantes puede ser aún más compleja.

A medida que aumenta el número de características, los puntos de datos se vuelven más dispersos y las métricas de distancia pierden información, ya que las distancias entre puntos son menos pronunciadas, lo que dificulta distinguir entre puntos cercanos y distantes. Esto se conoce como la *maldición de la dimensionalidad* .

La escasez de datos dificulta el entrenamiento de los modelos y los hace más propensos a sobreajustarse, capturando el ruido en lugar de los patrones subyacentes. Esto conduce a una generalización deficiente a datos nuevos e inéditos.

La reducción de dimensionalidad se utiliza en ciencia de datos y aprendizaje automático para reducir el número de variables o características en un conjunto de datos, conservando al máximo la información original. Esta técnica es útil para simplificar conjuntos de datos complejos, mejorar la eficiencia computacional y facilitar la visualización de datos.

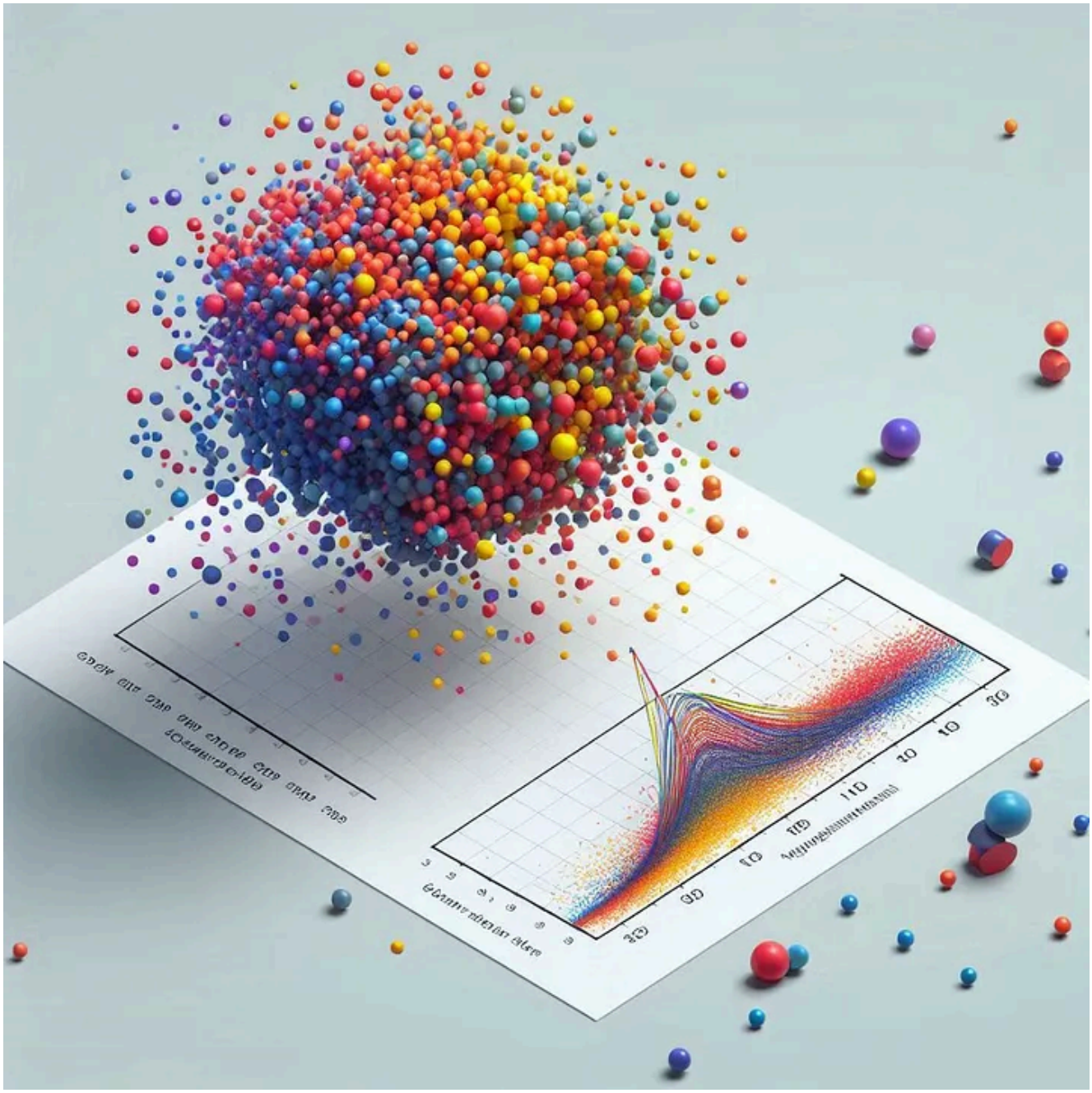


Imagen del autor utilizando DALL-E.

2. ¿Cómo funciona el análisis de componentes principales?

Una de las técnicas más utilizadas para mitigar el problema de la dimensionalidad es el Análisis de Componentes Principales (ACP). El ACP reduce el número de características en un conjunto de datos, conservando la mayor parte de la información útil al encontrar los ejes que representan la mayor varianza en el conjunto de datos. Estos ejes se denominan *componentes principales* .

Dado que PCA tiene como objetivo encontrar una representación de baja dimensión de un conjunto de datos manteniendo una gran parte de la varianza en lugar de realizar predicciones, se considera un algoritmo **de aprendizaje no supervisado** .

Pero ¿por qué mantener la varianza significa mantener información importante?

> Imagina que estás analizando un conjunto de datos sobre delitos en una ciudad. Los datos presentan numerosas características, como "delitos contra las personas con lesiones" y "delitos contra las personas sin lesiones". Sin duda, los lugares con altas tasas del primer ejemplo también deben tener altas tasas del segundo.

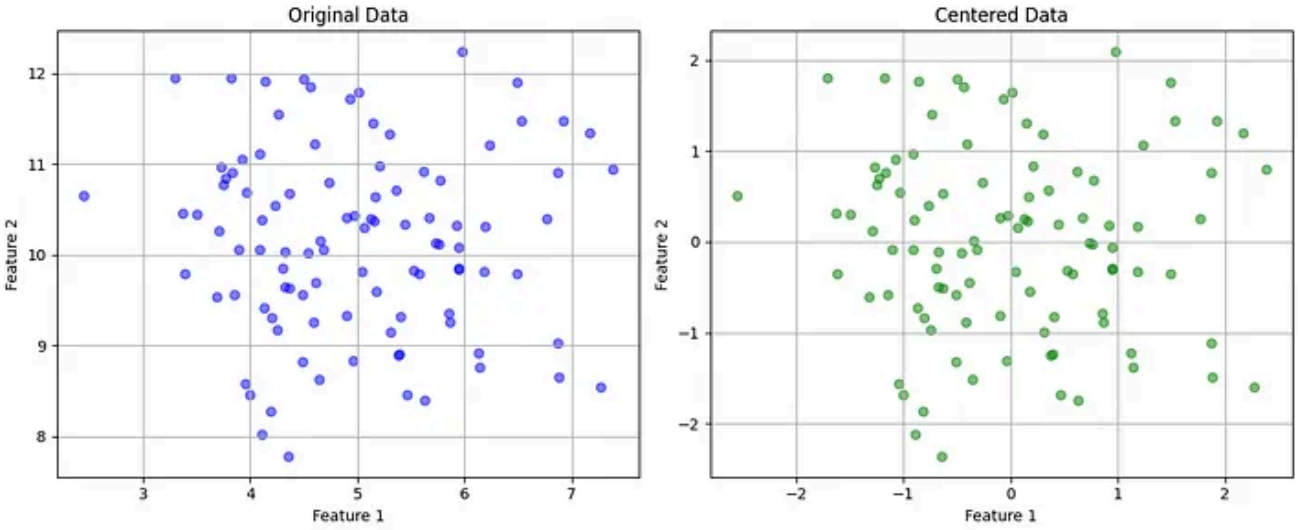
En otras palabras, las dos características del ejemplo están muy correlacionadas, por lo que es posible reducir las dimensiones de ese conjunto de datos disminuyendo las redundancias en los datos (la presencia o ausencia de lesiones en la víctima).

El algoritmo PCA no es más que una forma sofisticada de hacerlo.

Ahora, analicemos cómo funciona el algoritmo PCA en los siguientes pasos:

Paso 1: Centrar los datos

El PCA se ve afectado por la escala de los datos, por lo que lo primero que hay que hacer es restar la media de cada característica del conjunto de datos, garantizando así que todas las características tengan una media igual a 0.

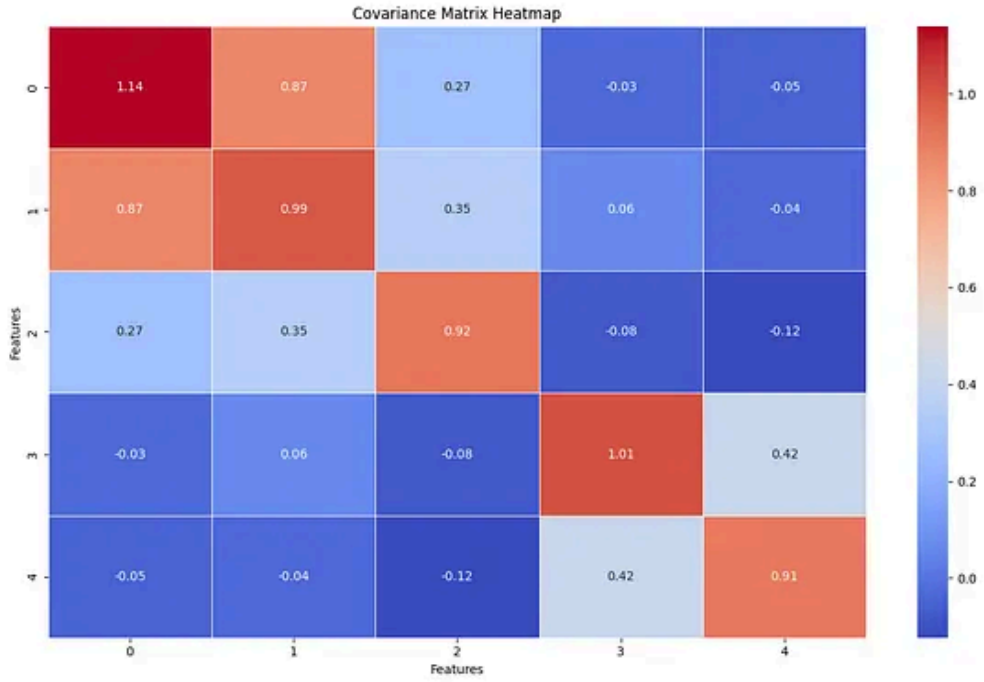


Datos antes y después del centrado (imagen del autor).

Paso 2: Cálculo de la matriz de covarianza

Ahora, debemos calcular la matriz de covarianza para capturar cómo varía conjuntamente cada par de características de los datos. Si el conjunto de datos tiene n características, la matriz de covarianza resultante tendrá forma n x n .

En la imagen de abajo, las características más correlacionadas tienen colores más cercanos al rojo. Por supuesto, cada característica estará altamente correlacionada consigo misma.



Mapa de calor de la matriz de covarianza (imagen del autor).

Paso 3: Descomposición de valores propios

A continuación, debemos realizar la descomposición en valores propios de la matriz de covarianza. Por si no lo recuerda, dada la matriz de covarianza Σ (una matriz cuadrada), la descomposición en valores propios consiste en encontrar un conjunto de escalares (valores propios) y vectores (vectores propios) tales que:

$$\Sigma v = \lambda v$$

Propiedad de valor propio (imagen del autor utilizando codecogs).

Dónde:

- Σ es la matriz de covarianza n×n.
- v es un vector distinto de cero llamado vector propio.
- λ es un escalar llamado valor propio asociado con el vector propio v .

> ***Vectores propios****indican las direcciones de máxima varianza en los datos (los componentes principales), mientras que****valores propios****cuantificar la varianza capturada por cada componente principal.*

Si una matriz **A** se puede descomponer en valores propios y vectores propios, se puede representar como:

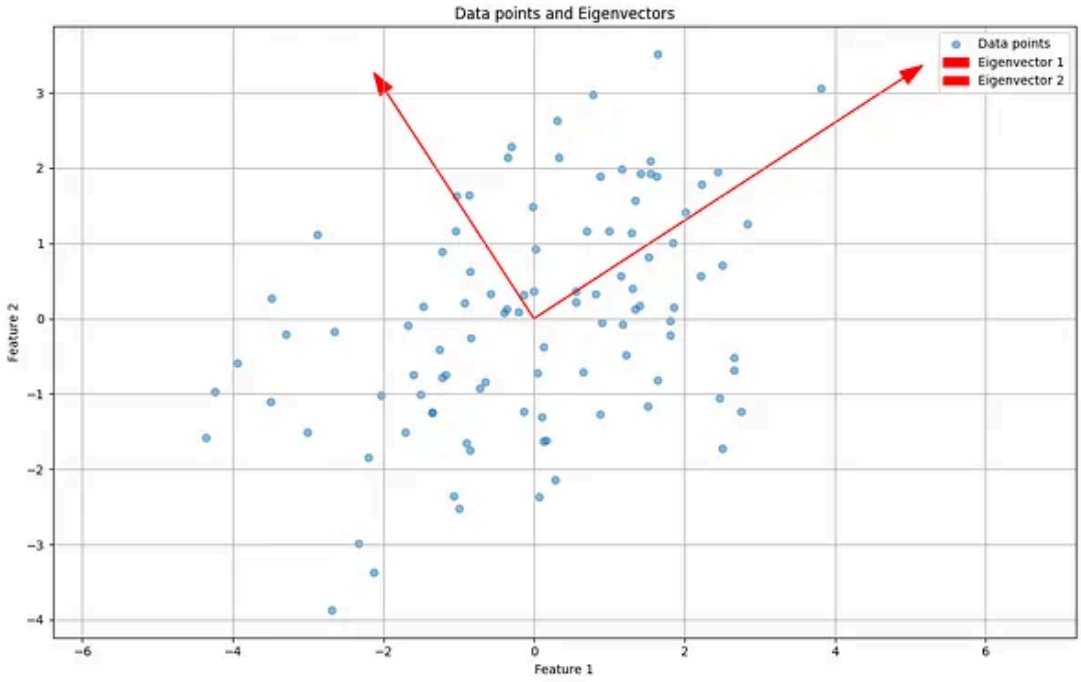
$$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^{-1}$$

Descomposición propia de una matriz (imagen del autor utilizando codecogs).

Dónde:

- **Q** es una matriz cuyas columnas son los vectores propios de **A.**
- Λ es una matriz diagonal cuyos elementos diagonales son los valores propios de **A** .

De esta manera, podemos utilizar los mismos pasos para encontrar los valores propios y los vectores propios de la matriz de covarianza.



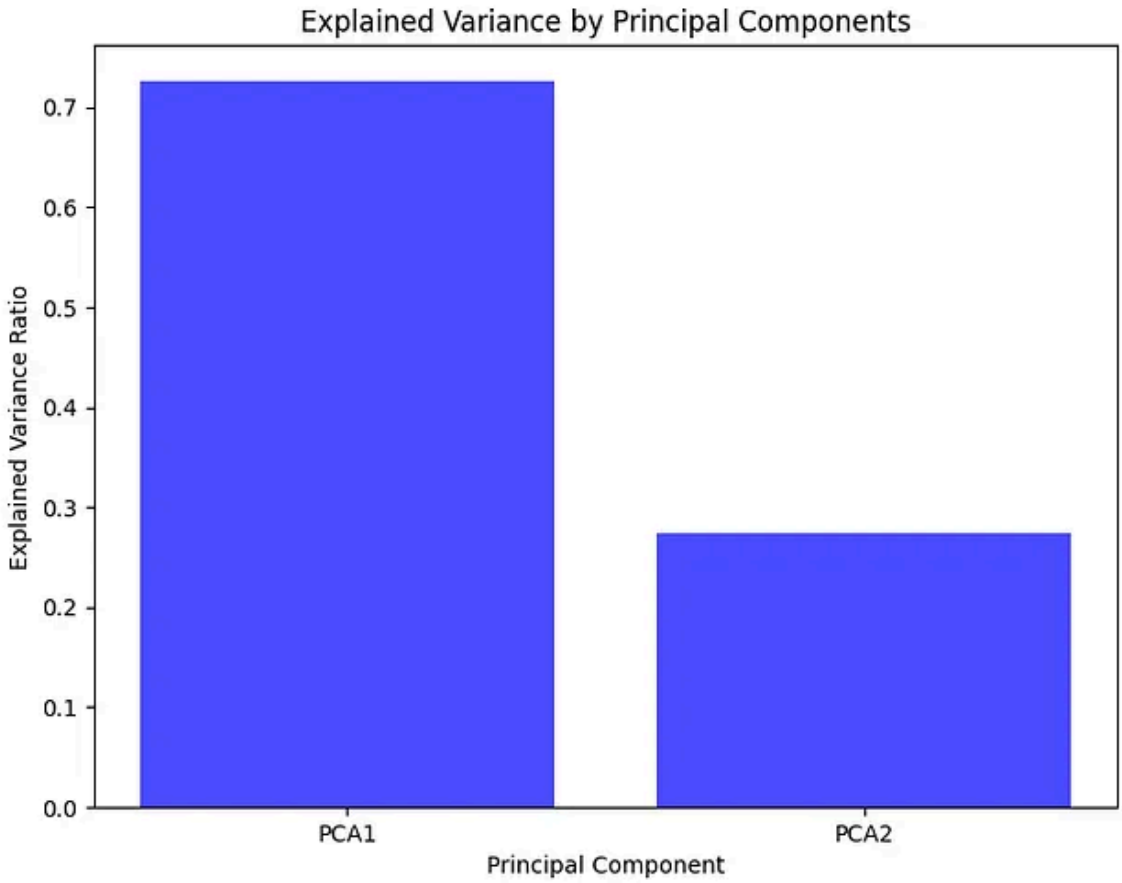
Trazando vectores propios (imagen del autor).

En la imagen de arriba, podemos ver que el primer vector propio apunta a la dirección con la mayor varianza de los datos, y el segundo vector propio apunta a la dirección con la segunda mayor varianza.

Paso 4: Selección de los componentes principales

Como se mencionó anteriormente, los autovalores cuantifican la varianza de los datos en la dirección de su vector propio correspondiente. Por lo tanto, ordenamos los autovalores en orden descendente y conservamos solo los n *componentes principales* requeridos .

La imagen a continuación ilustra la proporción de varianza capturada por cada componente principal en un PCA con dos dimensiones.



Varianza explicada para 2 componentes principales (imagen del autor).

Paso 5: Proyectar los datos

Finalmente, debemos proyectar los datos originales sobre las dimensiones representadas por los componentes principales seleccionados. Para ello, debemos multiplicar el conjunto de datos, una vez centrado, por la matriz de vectores propios obtenida al descomponer la matriz de covarianza.

$$X_{proj} = X \cdot \mathbf{Q}$$

Proyectando el conjunto de datos original a n dimensiones (imagen del autor usando codecogs).

3. Implementación en Python

Ahora que entendemos profundamente los conceptos clave del análisis de componentes principales, es hora de crear algo de código.

Primero, tenemos que configurar el entorno importando el paquete numpy para los cálculos matemáticos y matplotlib para la visualización:

```
importar numpy como np
importar matplotlib.pyplot como plt
```

A continuación, encapsularemos todos los conceptos cubiertos en la sección anterior en una clase Python con los siguientes métodos:

- **método init**

Método constructor para inicializar los parámetros del algoritmo: el número de componentes deseados, una matriz para almacenar los vectores de los componentes y una matriz para almacenar la varianza explicada de cada dimensión seleccionada.

- **método de ajuste**

En el método de ajuste, se implementan mediante código los cuatro primeros pasos presentados en la sección anterior. Además, se calculan las varianzas explicadas de cada componente.

- **Método de transformación**

El método de transformación realiza el último paso presentado en la sección anterior: proyectar los datos en las dimensiones seleccionadas.

- **Trazar la varianza explicada**

El último método es una función auxiliar para trazar la varianza explicada de cada componente principal seleccionado como un gráfico de barras.

Aquí está el código completo:

```
clase  PCA :
    def  __init__ ( self, n_components ):
        self.n_components = n_components
        self.components = None
         self.mean = None
         self.explained_variance = None

    def  fit ( self, X ):
        # Paso 1: Estandarizar los datos (restar la media)
         self.mean = np.mean(X, axis= 0 )
        X_centered = X - self.mean

        # Paso 2: Calcular la matriz de covarianza
         cov_matrix = np.cov(X_centered, rowvar= False )

        # Paso 3: Calcular los valores propios y los vectores propios
         eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

        # Paso 4: Ordenar los valores propios y los vectores propios correspondi
         sorted_indices = np.argsort(eigenvalues)[::- 1 ]
        eigenvalues = valores propios[índices_ordenados]
        vectores propios = vectores propios[:, índices_ordenados]

        # Paso 5: Seleccionar los n_componentes superiores
         self.components = vectores propios[:, :self.n_components]

        # Calcular la varianza explicada
         total_variance = np. suma (valores propios)
        self.varianza_explicada = valores propios[:self.n_componentes] / varianz

    def  transform ( self, X ):
        # Paso 6: Proyectar los datos en los componentes seleccionados
         X_centrado = X - self.mean
        return np.dot(X_centrado, self.componentes)

    def  plot_varianza_explicada ( self ):
        # Crear etiquetas para cada componente principal
         labels = [ f'PCA {i+ 1 } '  for i in  range (self.n_componentes)]

        # Crear un gráfico de barras para la varianza explicada
         plt.figure(figsize=( 8 , 6 ))
        plt.bar( range ( 1 , self.n_componentes + 1 ), self.varianza_explicada,
        plt.xlabel( 'Componente principal' )
        plt.ylabel( 'Razón de varianza explicada' )
        plt.title( 'Varianza explicada por componentes principales' )
        plt.show()
```

4. Evaluación e interpretación

Ahora es el momento de usar la clase que acabamos de implementar en un conjunto de datos simulado creado con el paquete numpy. El conjunto de datos tiene 10 características y 100 muestras.

```
# crear datos simulados para análisis
 np.random.seed( 42 )
# Generar una señal de baja dimensión
 low_dim_data = np.random.randn( 100 , 4 )

# Crear una matriz de proyección aleatoria para proyectar en dimensiones mayores
 projection_matrix = np.random.randn( 4 , 10 )

# Proyectar los datos de baja dimensión a dimensiones mayores
 high_dim_data = np.dot(low_dim_data, projection_matrix)

# Agregar algo de ruido a los datos de alta dimensión
 noise = np.random.normal(loc= 0 , scale= 0.5 , size=( 100 , 10 ))
data_with_noise = high_dim_data + noise

X = data_with_noise
```

Antes de realizar el ACP, queda una pregunta: **¿cómo elegimos el número correcto u óptimo de dimensiones** ? Generalmente, debemos buscar el número de componentes que sumen al menos el 95 % de la varianza explicada del conjunto de datos.

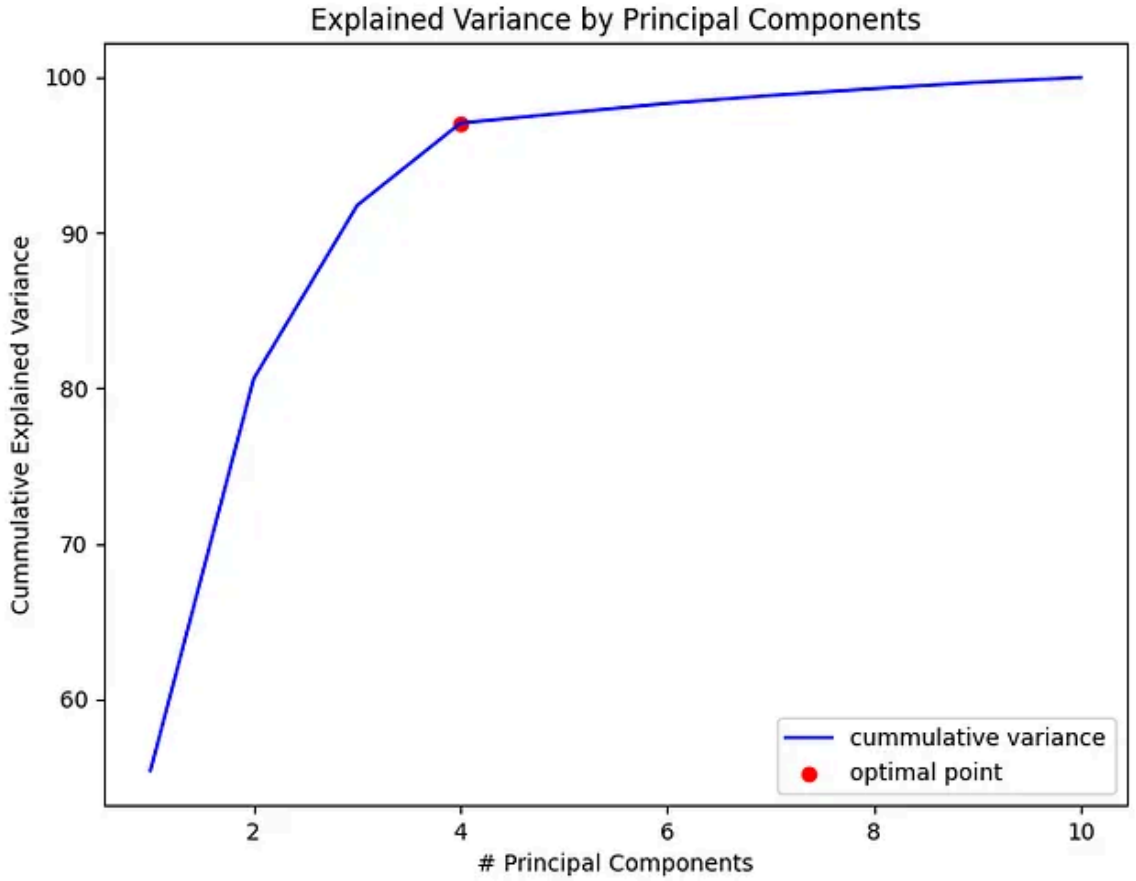
Para ello, veamos cómo cada componente principal contribuye a la varianza total del conjunto de datos:

```
# Aplicar PCA
 pca = PCA(n_components= 10 )
pca.fit(X)
X_transformed = pca.transform(X)

print ( "Varianza explicada:\n" , pca.explained_variance)
```

```
>> Varianza explicada (%):
[ 55.406 , 25.223 , 11.137 , 5.298 , 0.641 , 0.626 , 0.511 , 0.441 , 0.401 , 0.3
```

A continuación, grafiquemos la suma acumulada de la varianza y verifiquemos en qué número de dimensiones logramos el valor óptimo del 95% de la varianza total.



Varianza explicada en función del número de componentes (imagen del autor).

Como se muestra en el gráfico anterior, el número óptimo de dimensiones para el conjunto de datos es 4, lo que representa el 97,064 % de la varianza explicada. En otras palabras, transformamos un conjunto de datos con 10 características en uno con solo 3 dimensiones, conservando más del 97 % de la información original.

> Esto significa que la mayoría de las 10 características originales estaban muy correlacionadas y el algoritmo transformó esos datos de alta dimensión en componentes principales no correlacionados.

5. Conclusiones y próximos pasos

Creamos una clase PCA usando solo el paquete numpy que redujo exitosamente la dimensionalidad de un conjunto de datos de 10 características a solo 4 mientras preservaba aproximadamente el 97% de la varianza de los datos.

Además, exploramos un método para obtener un número óptimo de componentes principales del análisis PCA que se puede personalizar dependiendo del problema al que nos enfrentamos (podríamos estar interesados en retener sólo el 90% de la varianza, por ejemplo).

Esto demuestra el potencial del análisis PCA para abordar la maldición de la dimensionalidad, explicada anteriormente. Además, me gustaría dejar algunos puntos para mayor exploración:

- Realice tareas de clasificación o regresión utilizando otros algoritmos de aprendizaje automático en el conjunto de datos reducido utilizando el algoritmo PCA y compare el rendimiento de los modelos entrenados en el conjunto de datos original frente al conjunto de datos transformado por PCA para evaluar el impacto de la reducción de la dimensionalidad.
- Utilice PCA para la visualización de datos para hacer que los datos de alta dimensión sean más interpretables y para descubrir patrones que no eran evidentes en el espacio de características original.
- Considere explorar otras técnicas de reducción de dimensionalidad, como *la incrustación estocástica de vecinos distribuida en t* (t-SNE) y *el*

análisis discriminante lineal (LDA).

Código completo disponible aquí .



. . .

Siéntete libre de usar y mejorar el código, comentar, hacer sugerencias y conectarte conmigo en LinkedIn , X y Github .

Referencias

[1] Willmott, Paul. (2019). *Aprendizaje automático: una introducción a las matemáticas aplicadas* . Panda Ohana Publishing.

[2] Géron, A. (2017). *Aprendizaje automático práctico* . O'Reilly Media Inc.

[3] Grus, Joel. (2015). *Ciencia de datos desde cero* . O'Reilly Media Inc.

[4] Datagy.io. *Cómo realizar PCA en Python* . Recuperado el 2 de junio de 2024 de https://datagy.io/python-pca/ .

Aprendizaje automático | Ciencia de datos | Reducción de dimensionalidad